

121 GROUP · CALCULATIONS REFERENCE

How everything is calculated

How every metric, status, and rollup in the app is calculated. Written for the portal dev (so the same numbers land in both builds), the team (so we all agree on what "On Fire" means), and leadership (so the numbers in the report match what's on screen).

Each entry: plain-English summary, formal rule, where it surfaces in the UI, and the source file in code.

A reference for the dev (so the portal calculates the same numbers our build does), team members (so we all agree on what "On Fire" means), and leadership (so the numbers in the report match what's on screen).

Every metric, status, and rollup the app surfaces is listed here. For each one: a plain-English explanation, the formal rule, where it shows up in the UI, and the source file in code. Each entry also carries a **How it should be** section — Nikko's recommendation for the model going forward. Some items recommend "Keep as is" (the current rule is the right one); others propose meaningful changes.

TL;DR

Two flavors of values in the app:

- **Stored values.** The AM (or another operator) picks them. The system never overrides what a person chose. Examples: each client's status (fire / risk / track / onboard / paused), each service's progress percentage, each task's severity.
- **Derived values.** Computed live from the underlying data. The system recalculates on every render — no cached or denormalized copies sit on records. Examples: each service's health, every count in NEEDS ATTENTION, the Portfolio Health rollup, day-card load.

When the portal disagrees with our build's numbers, the cause is almost always one of two things: either the portal is using a different *derivation rule*, or it's storing a value the reference treats as derived (or vice versa).

The direction of travel (covered in the "How it should be" sections): the model should move toward **auto-derive by default with a manual override flag** for the values that today are fully manual. The override flag preserves AM control while letting the system catch drift.

1. Status vocabulary

The app uses one canonical set of status names everywhere. Five client-level states, three service-level states.

Client statuses

Status	Meaning	What an AM does with it
fire ("On Fire")	The client needs hands-on attention right now. Open tasks are overdue, blocked, or critical.	Open the client, triage the loudest task, get the blocker moving.
risk ("At Risk")	Work is drifting. Not on fire yet, but reviews are piling up or schedule is slipping.	Check in this week. Likely a half-hour conversation.
onboard ("Onboarding")	Brand-new client still inside the first-30-days setup window.	Run the onboarding checklist. Don't expect production output yet.

track ("On Track")	Quietly working. No urgent signal.	Leave alone. The system surfaces them only when something changes.
paused	Client is on hold (renewal in flight, payment dispute, vacation freeze). Doesn't count toward "active clients."	Resume when the blocker clears.

archived is a separate axis — an archived client is filtered out of every list by default and only resurfaces when the user explicitly opens the Archived filter chip.

Storage model: each client carries a status field. The AM sets it manually via the Add Client modal and via the kebab menu on the client detail header. The system never auto-changes it. Type definition: `src/types/flizow.ts:22 (ClientStatus)`.

How it should be: Auto-derive by default — `client.status = worst(service.health for service in client.services)` with the priority order `fire > risk > track`. `onboard` and `paused` stay manual (they're operational states, not health states). When the AM disagrees with the derived value, they can flip to a manual override with a visible "Manually set by [name] on [date]" indicator that stays until cleared. Solves the drift problem: a client whose services are all on fire shouldn't read as On Track just because nobody updated the field. The override flag preserves AM agency while the system carries the boring vigilance.

Why the current model causes drift (concrete example):

A real case that surfaced during QA. The AM flagged a task as "urgent priority" and the `dueDate` had already passed (1 day overdue). The MY TASKS card pill rendered as **AT RISK**, not **ON FIRE** — which felt wrong because urgent overdue work is the loudest possible signal.

The chain explains why:

```
Task: "Review prospect's website copy & messaging"
■ ■ dueDate: yesterday → overdue
■ ■ severity: 'critical' (urgent flag)
■ ■ columnId: in-progress (not blocked)
■
▼ Service health (DERIVED from tasks)
Service: any task overdue OR critical OR blocked → service = 'fire'
■
▼ Client status (MANUAL – not derived)
Client: status stays at 'risk' because no AM has flipped it manually
■
▼ MY TASKS card pill
Pill = client.status = AT RISK ← reads the manual field, not the
actual ground-truth state
```

The card pill reads `client.status` (manual). The service underneath is technically on fire by the derivation rule, but the client record one level up is still hand-marked as at-risk. There's no auto-promotion path from "service went on fire" → "client status gets bumped to fire."

Worse: the MY TASKS feed only includes clients where `status === 'fire' || status === 'risk'`. So a client manually marked track (On Track) with urgent overdue work **wouldn't appear in MY TASKS at all** — silently invisible to whoever is scanning the dashboard.

The auto-derive fix below solves both problems at once.

Implementation blueprint (for the portal dev):

Step 1 — Data model. Add two fields to the Client record alongside the existing status:

```
interface Client {  
  // existing  
  status: ClientStatus;  
  
  // new – manual override system  
  statusOverride?: ClientStatus | null;  
  statusOverrideBy?: string | null; // member id of whoever set it  
  statusOverrideAt?: string | null; // ISO timestamp  
}
```

The existing status field stays but its meaning shifts: it now seeds the initial value (e.g. 'onboard' when a client is created) and stores the operational state (paused / onboard). The single source of truth for the displayed status becomes the derivation function below.

Step 2 — Derivation function. One pure function, runs on every render:

```
function deriveClientStatus(  
  client: Client,  
  services: Service[],  
  tasks: Task[],  
  todayISO: string,  
) : ClientStatus {  
  // 1. Manual override always wins.  
  if (client.statusOverride) return client.statusOverride;  
  
  // 2. Operational states are sticky (not health-driven).  
  if (client.status === 'paused' || client.status === 'onboard') {  
    return client.status;  
  }  
  
  // 3. Derive from worst service health across the client's services.  
  const ownedServices = services.filter(s => s.clientId === client.id);  
  if (ownedServices.length === 0) return 'track';  
  
  let worst: ClientStatus = 'track';  
  for (const service of ownedServices) {  
    const health = serviceHealth(service.id, tasks, todayISO);  
    if (health === 'fire') return 'fire'; // short-circuit  
    if (health === 'risk') worst = 'risk';  
  }  
  return worst;  
}
```

```
}
```

Step 3 — Override behavior. When the AM manually picks a status via the client-detail kebab menu or the Add Client modal:

- Write to `statusOverride` (NOT `status`).
- Write `statusOverrideBy = currentMemberId` and `statusOverrideAt = new Date().toISOString()`.
- A "Use auto" button next to the manual indicator clears all three override fields back to null.
- The override does NOT auto-expire. If the AM disagrees, they keep the override until they explicitly clear it. Auto-expiry would surprise the AM ("why did this revert overnight?").

Step 4 — UI swap. Every place that reads `client.status` today swaps to `deriveClientStatus(client, services, tasks, today)`:

Surface	Today	After
Client detail header pill	<code>client.status</code>	derived value
MY TASKS aggregator	filter on <code>client.status</code>	filter on derived value
Portfolio Health rollup	count by <code>client.status</code>	count by derived value
Clients list filter chips	bucket by <code>client.status</code>	bucket by derived value
Clients list row dot color	<code>client.status</code>	derived value

Step 5 — "Manually set" indicator. When `statusOverride` is non-null on the surface being rendered, display a small caption under the status pill:

● ON TRACK
Manually set by Jane on May 10 · Use auto

Style: 12px muted text. The "Use auto" link clears the override. Renders only on surfaces where space allows (client detail header yes; Clients list row no — too narrow).

Step 6 — Migration. No data migration needed. Existing client records keep their `status` field as-is. On the first render after the deploy, `deriveClientStatus()` runs and either:

- Returns the existing `status` value (when the derived value happens to match) — no visible change for the AM.
- Returns a different value (when the manual status drifted from service health) — the pill updates to the new value. If the AM prefers the old value, they click the kebab → set status manually → it writes to `statusOverride` → the manual indicator appears.

The transition is graceful — no broken state, no missing data, the worst case is one AM seeing a status they didn't expect and choosing whether to keep it or override.

Step 7 — Edge cases to handle:

- **No services yet** → return 'track'. New client with no work shouldn't read as urgent.
- **All services archived** → archived services are filtered out before derivation runs. If everything is archived, return 'track'.
- **Mixed signals** — the priority order (fire > risk > track) handles this. One fire service trumps two track services.
- **Race conditions** — derivation is pure. Stale UI between a task move and a re-render isn't a problem; the next render computes the right value.
- **Paused / onboard precedence** — these win over derived health because they're operational, not health-driven. A paused client with one on-fire service should still read as paused until the AM un-pauses them.

Service statuses

Same three loudest names as clients (fire / risk / track), but service status is **derived** from tasks. There's no paused or onboard at the service level.

Status	Rule (in order)
fire ("On Fire")	Any open task is overdue (dueDate < today), marked critical severity, or sitting in the blocked column.
risk ("At Risk")	Any open task is marked warning severity (drifting work, reviews piling up). No fire signals present.
track ("On Track")	Everything else. Includes "no tasks yet" — a brand-new service shouldn't read as a problem.

"Open" excludes archived tasks and tasks already in the done column. The first fire signal short-circuits the loop (no need to keep checking other tasks once a fire flag is set).

Source: src/utils/clientDerived.ts:132 (serviceHealth).

How it should be: Keep as is. The derivation from tasks is the right shape — first urgent signal wins, no priors, no cache that can go stale. Service health is the foundation for the client-status auto-derive recommended above, so getting this rule pinned down is the cornerstone the rest builds on. Short-circuiting on the first fire signal stays.

2. Per-client values

Portfolio Health (Home page)

The three big metric cards at the top of Home — ON FIRE / AT RISK / ON TRACK.

Plain English: count of clients currently in each state. Paused clients aren't shown (they're not in active rotation).

Formal rule:

```

fire = count of clients where status === 'fire'
risk = count of clients where status === 'risk'
track = count of clients where status === 'track'
active = total clients minus paused

```

The "Across N active clients" line above the cards uses active. The clicks on each card route to /clients?view=<state> and the Clients filter chip with the same state shows the matching count — they share one source of truth.

Source: src/pages/OverviewPage.tsx:166 (health memo).

How it should be: Keep the count math. Once client status auto-derives from service health (per §1), this rollup automatically reflects ground-truth service state — no separate work needed. The "active = total – paused" formula stays right; paused clients shouldn't pad the active rotation.

Subtitle ("X clients need you now" / "Steady morning ahead.")

Adaptive copy below the greeting:

- If fire > 0: "X clients need you now."
- Else if there's overdue work: "X tasks overdue across the workspace."
- Else: a time-of-day greeting fallback ("Steady morning ahead.", etc.)

The branch table lives in pickTagline (same file).

How it should be: Keep the adaptive branching. Three states (loud / mid / calm) is the right level of resolution. Add one more branch at the bottom: when fire = 0 AND overdue = 0 AND there are quiet/drift clients (see §5), surface that as a fourth state — "X clients haven't been touched in 2 weeks." Catches the silent drift case the urgency model misses.

Client list row — right-hand metric

Each row carries a short status-aware sentence on the far right. The sentence shape depends on the client's status:

Status	Metric text
fire	{n} overdue (e.g. "5 overdue"). Falls back to "Needs attention" if there's no overdue task yet.
risk	{n} at risk — count of tasks with severity warning or critical. Falls back to "At risk".
onboard	{N}% setup — average progress across the client's services.
paused	"Paused"
track (or anything else)	"On track"

Source: src/utils/clientDerived.ts:79 (clientMetric).

How it should be: Keep the status-aware sentence shape. Add one more state once the drift detector lands: clients with `daysSinceLastActivity > 14` AND no urgent signal show "Quiet — last touched X days ago" instead of "On track." Surfaces the silent-drift case the loud states miss. Same right-aligned slot, same render rule, one more branch.

Last-activity timestamp

The "6d ago" / "Mon" / "Apr 7" stamp on the right of each list row.

Plain English: the most recent activity touching this client. Today, in the demo data, we use the latest `createdAt` of any task tied to the client's services. Falls back to the client's `startedAt` so a brand-new client with no tasks yet still has a date.

Source: `src/Utils/clientDerived.ts:188 (clientLastTouched)`.

How it should be: Add a proper `updatedAt` field on each client record. Write to it on every operator-driven mutation: task add/move/comment, note add, contact change, quick-link change, onboarding tick, service add, client field edit, touchpoint logged. Drop the demo's "latest task.createdAt" walk — it's slow at scale (walks every task) and misses non-task activity (notes, onboarding, contacts, touchpoints). Keep the format ladder (§ below) unchanged; just feed it a real timestamp.

"Xd ago" formatting

The same formatter renders the timestamp in three buckets:

Age	Format
Under 1 minute	"just now"
Under 1 hour	"Xm ago" (minutes)
Under 1 day	"Xh ago" (hours)
Under 7 days	"Xd ago" (days)
Under 30 days	weekday name ("Mon", "Tue") — friendlier than "14d ago"
30+ days	short date ("Apr 7")

Anchored against the store's today value (not `Date.now()`) so the output stays stable across re-renders.

Source: `src/Utils/clientDerived.ts:205 (relativeTimeAgo)`.

How it should be: Keep the format ladder. The buckets (minutes / hours / days / weekday / date) match how humans actually think about recency. Anchoring on the store's today (not `Date.now()`) is also right — stable across re-renders, deterministic for tests.

3. Per-service values

Service type

Two types: project (one-off deliverable that ships once) and retainer (recurring monthly scope). Stored on the service record; the AM picks it in the Add Service modal.

Source: `src/types/flizow.ts:35 (ServiceType)`.

How it should be: Keep the project/retainer split. Two types covers the meaningful distinction (one-off vs recurring). Anything finer (sprint, audit, content series, growth experiment) becomes a template under one of the two types, not a third type. Three types would force a label switch in too many surfaces (progress label, subtitle shape, retainer-specific UI) for diminishing return.

Service health

Derived from tasks (rule listed in §1 above). Surfaces as:

- A colored dot or pill on each service row in ACTIVE SERVICES
- The colored health bar on the pinned card on Home (red / amber / green)

How it should be: Covered in §1 above. Keep the derivation rule. This is the calculation the rest of the model leans on (client status auto-derive, NEEDS ATTENTION, MY TASKS aggregation) — keeping it pure-derived (no cache) means changes propagate instantly.

Service progress

Plain English: 0–100 number that drives the progress bar on the service card.

Storage model: stored on the service record as progress. **Not** derived from task completion. Whoever owns the service updates it manually as work moves.

Source: `src/types/flizow.ts:394 (Service.progress)`.

How it should be: Switch to auto-compute by default, with a manual override flag:

- `project: progress = (tasks where columnId === 'done' and !archived) / (total open + done tasks) × 100`
- `retainer: progress = (hours logged this month) / (hours budgeted this month) × 100` — requires a `hoursBudgeted` field on retainer services + a time-tracking integration or a manual hours field

Manual override (`progressOverride: number | null`) plus a "Manually set" indicator. Today's manual model goes stale fast — AMs forget to update the number for weeks while the work moves. Auto-compute keeps it honest; override lets the AM correct when the count misleads (the last 20% taking 50% of the time is real, and the AM judging that is fine, but the default should be the count).

Service "Next deliverable" date

Stored on the service record as `nextDeliverableAt`. The AM sets it in the Add Service modal and updates it as milestones move. Drives the "Due May 25" subtitle line on each service row in ACTIVE

SERVICES (see §4).

Source: `src/types/flizow.ts:396`.

How it should be: Derive from the earliest open task in a designated "milestone" lane on the service's kanban (or earliest open task overall if no milestone lane is designated). Manual override allowed via a `nextDeliverableOverride` field. Same drift problem as progress: today's manual model goes stale because nobody updates the field as new tasks land. Pull the next checkpoint from the actual board state and let the AM override only when the ground-truth date differs.

ACTIVE SERVICES eyebrow ("7 of 7 · 2 projects, 5 retainers")

The count + split line under ACTIVE SERVICES on the client detail.

```
total = services.length
projects = services.filter(s => s.type === 'project').length
retainers = services.filter(s => s.type === 'retainer').length
eyebrow = "{total} of {total} · {projects} projects, {retainers} retainers"
```

Note on the "X of X" prefix: right now both numbers are the services-array length. The prefix exists because the original spec wanted "active of total" (excluding archived/paused services) but archive at the service level isn't surfaced in this UI yet, so they're equal.

Source: `src/pages/ClientDetailPage.tsx:904`.

How it should be: Once archive-at-service-level lands, the prefix becomes `active of total` where `active = services.filter(s => !s.archived).length`. Already structurally wired (both numbers exist); only the UI exposure of archive is missing. The project/retainer split stays as is — the breakdown is useful at a glance even on small service counts.

Service row subtitle ("Template: Brand Refresh · Due May 25")

Two parts separated by a middle dot:

- **Template:** the service's `templateKey` resolved to its display name. If no template assigned: "Template: None".
- **Due:** `nextDeliverableAt` formatted as a short date.

For retainers, the subtitle still shows the template + next deliverable date — the "next deliverable" on a retainer is the next monthly checkpoint.

How it should be: Keep the two-piece shape (Template · Due). It's the right pair of context cues (what kind of work + next checkpoint). Replace "Template: None" with "Untemplated service" when no template is assigned — clearer signal that the operator skipped templating intentionally rather than the data being broken. "Due —" placeholder on services with no scheduled next deliverable should fall back to "No milestone scheduled."

Service progress label ("Progress 35%" vs "This month 33%")

Same numeric value, different label depending on service type:

- project → label reads "Progress" (cumulative completion)
- retainer → label reads "This month" (current-month consumption)

The label switch is purely cosmetic — both pull from `service.progress`. The framing differs because the same number *means* something different for the two service types: 35% complete on a one-off deliverable vs. 35% of this month's retainer hours used.

How it should be: Keep the label switch. Once the auto-compute math lands (see Service progress above), the This-month number gets honest — derived from real hours used this month rather than a manually-typed number. Same label, more trustworthy number. No UI change needed.

4. Per-task values

Overdue

`task.dueDate < today` AND the task isn't archived AND isn't in the done column. Comparison is lexicographic on ISO date strings — works correctly without parsing because the format is sortable.

Source: `src/utils/clientDerived.ts:164 (countOverdueTasks)`.

How it should be: Keep the rule. `task.dueDate < today` with lexicographic comparison on ISO strings is cheap, correct, and parses-free. Already excludes done + archived. No change.

Blocked

`task.columnId === 'blocked'`. The blocked column is a fixed lane on every kanban board.

How it should be: Add a time-based escalation. Any task sitting in `blocked` for more than 5 days auto-flags `severity === 'critical'` (if not already set). The "blocked" state today is silent — only the AM eyeballing the board catches that work has been stuck for two weeks. Auto-promotion to critical surfaces it in NEEDS ATTENTION, MY TASKS, and the client metric without anyone having to remember to mark it.

Severity (warning / critical)

A manually-set field on each task. The owner marks it via the card modal. Drives:

- Service-health derivation (any `critical` → service goes On Fire; any `warning` → service goes At Risk)
- Client list row metric ("N at risk" counts both `warning` and `critical`)

The system never auto-sets severity — it's the operator's read on the work.

How it should be: Keep severity as a manual field, BUT add a structured `blockingCategory` enum alongside the free-text blocker reason:

```
blockingCategory: 'budget' | 'client-response' | 'asset-delivery'
| 'dependency' | 'feedback' | 'other'
```

Free-text caption stays for the human context. The structured field unlocks Stats-tab insights ("60% of blockers this quarter were waiting on client response — fix the asset request process") and a reporting dimension for leadership that the free-text field can't serve.

5. MY TASKS card aggregation (Home)

This section renders one card per fire-or-risk client (not one card per task). Cards are sorted by severity, then by total urgent task count, then by oldest overdue date.

For each card in the feed:

Field	Rule
Severity pill	<code>client.status</code> rendered as "On Fire" or "At Risk".
Client name	<code>client.name</code> .
Aggregate title	<code>{N overdue} overdue · {M blocked} blocked</code> if both > 0 . Just the non-zero half if only one. Falls back to "Marked on fire — no blocker logged yet" for fire clients with zero urgent tasks.
Blocking-reason caption	The longest blocker reason on any task tied to the client. Falls back to the primary task title when no blocker reason exists.
Days-overdue stamp (right)	Anchored on the oldest overdue task: <code>floor((today - oldestDueDate) / 86,400,000)</code> days. "1 day overdue" / "N days overdue". Falls back to "Blocked" / "Needs review" when nothing is measurably late.
Review button click	Deep-links to the kanban board of the primary task (worst-case first: oldest overdue → first blocked → client detail).
Delegate button click	Opens the Delegate popover anchored to that card.

Empty state: when `health.fire === 0` and no risk clients have unresolved tasks, the section renders "Nothing urgent right now. Enjoy the quiet."

Source: `src/pages/OverviewPage.tsx:1137 (buildAttentionCards)`.

How it should be: Two additions on top of the current model:

- Secondary view: by reason.** Add a toggle at the top of MY TASKS — "By client" (today's default) vs "By blocker." The by-blocker view groups tasks across clients by `blockingCategory` (see §4): "Waiting on client response (3)" / "Budget approval stalled (2)" / "Reviews overdue (4)." Same data, different cut, optimized for batch-work (the AM can plow through all the review-pending cards together).

- 2 **Drift section underneath.** Below the urgent cards, a small list: "X clients you haven't touched in 14+ days." Catches the silently-stale case. Each row is a one-liner with the client name + last-touched stamp + a "Open" button. Doesn't compete with the loud cards — it's a quiet reminder underneath them.

6. NEEDS ATTENTION (Client detail Overview tab)

Two-card block at the top of the Overview tab, computed from the client's open tasks. Each card renders only when its count > 0.

Card	Rule
Past-due card (red flame icon)	Count of open tasks where <code>severity === 'critical'</code> OR <code>columnId === 'blocked'</code> . Subtitle: name of the first such service + "tap to open". Clicks straight through to that service's board.
At-risk card (gray chat icon)	Count of open tasks where <code>severity === 'warning'</code> . Subtitle: name of the first such service + "review soon". Clicks straight through to that service's board.

Timestamp ("As of this morning") signals these are computed at start of day, not refreshed live.

Source: `src/pages/ClientDetailPage.tsx:783` (`buildAttentionChips`).

How it should be: Keep the two-card pattern. Once `blockingCategory` lands (§4), consider a third card type when the signal warrants it: "Waiting on client — N cards" (any blocked tasks where category is `'client-response'` or `'asset-delivery'`). Surfaces externally-blocked work separately from internally-blocked work so the AM knows whether to chase the team or chase the client. Renders only when count > 0.

7. Onboarding rollup (Client detail Onboarding tab)

The progress line on the right of "SETUP & ONBOARDING".

Plain English: how far along are this client's services in their template-driven onboarding checklists.

Formal rule:

```
totalItems = count of onboarding items across all the client's services
doneItems = count of items where done === true
openServices = count of services where doneCount < totalItemsForThatService

# Eyebrow string:
if openServices === 0:
  "All set · {doneItems} of {totalItems} items complete"
else:
```

```
"{openServices} of {servicesCount} services in progress · {doneItems} of {totalItems} items complete"
```

Each service onboarding card also carries its own subtitle showing the template name + items left (e.g. "Brand Refresh · 3 items left").

Source: `src/pages/ClientDetailPage.tsx:1283`.

How it should be: Keep the rollup math. Add one behavior: when a service hits "All set" for more than 7 days, collapse its onboarding card into a "Completed setups (N)" expandable group at the bottom of the tab. Stops the Onboarding tab from accreting a tail of done-but-still-listed services. The 7-day delay is a grace window so the AM has time to verify completeness before the card hides.

8. Capacity / daily slot load

The "0/6" badge on each day card in MY SCHEDULE, and the green / amber / red zone color.

Load

```
load = sum of (task.slots ?? 1) for every "open" task where the member is the assigneeId AND the task's dueDate matches
```

"Open" excludes archived tasks AND tasks already in the done column — finished work doesn't keep eating capacity.

Multi-owner tasks (`assigneeIds[]`) do NOT contribute to anyone's load. Only the primary `assigneeId` absorbs the slots. If two designers share a job, the AM splits it into two tasks. This intentional rule keeps "who owns this" unambiguous.

Source: `src/utils/capacity.ts:58 (loadFor)`.

How it should be: Keep the slot math. The primary-assignee-only rule is right — multi-owner contribution would double-count slots and break capacity planning. Add a fragmentation indicator alongside the load badge in MY SCHEDULE: "4/6 · 3 clients" surfaces switching cost (4 slots across 3 different clients is harder than 4 slots on one client). Same badge footprint, new secondary number.

Soft cap and Max cap

Two numbers per (member, date) pair, with a resolution chain:

1. Per-day override exists for (member, date) → use it
2. Else member has standing `capSoft/capMax` → use those
3. Else fall back to `DEFAULT_CAP_SOFT (6)` and `DEFAULT_CAP_MAX (8)`

Soft cap is the comfortable daily target. **Max cap** is the hard ceiling. The per-day cap-edit popover (the kebab on each day card) sets both numbers for a single day — overriding the standing defaults. A holiday or PTO day is modeled by overriding both to 0.

Source: `src/utils/capacity.ts:88 (effectiveCapFor)`. Defaults at lines 29, 33.

How it should be: Keep the resolution chain (override → standing → defaults). Add one auto-override: when a member has approved time-off for a date, automatically set soft+max to 0 on that date (currently the AM has to do this manually via the popover). PTO is the most common reason for an override; deriving it from the time-off system removes a chore and prevents the case where someone forgets to update the cap before going on leave.

Load zone (green / amber / red)

```
load <= soft → green (under target, quiet state)
soft < load <= max → amber (warning)
load > max → red (over capacity)
```

The zone drives the badge color on the day card and the heatmap shading on the Team Capacity Heatmap.

Source: `src/utils/capacity.ts:108` (`loadZone`).

How it should be: Keep the three-zone rule. Optionally add a fourth "deep red" zone for $\text{load} > 1.5 \times \text{max}$ — catastrophic over-booking that needs a separate visual signal so the AM doesn't miss it among normal red days. Polish, not blocking.

9. Service tag chips on the client list row

The three pills under each client name on the Clients page (e.g. "Brand Refresh Q2", "Marketing Site v3", "Conversion Optin").

```
ordered = resolve client.serviceIds into Service records,
in the order the client lists them
if ordered.length <= 3:
  visible = ordered service names
  overflow = 0
else:
  visible = first 3 service names
  overflow = ordered.length - 3 (rendered as "+N more")
```

Newest project work shows first because the demo data unshifts new projects to the front of `client.serviceIds`. Production should do the same.

Source: `src/utils/clientDerived.ts:46` (`servicePills`).

How it should be: Keep 3-visible + "+N more." Cap is right — a row with 8 chips is unscannable. Consider sorting by service health instead of creation order: ON FIRE services bubble to the visible slots so the worst signals are always seen, not buried under "+5 more." Useful when a client has many services and the AM scans the list looking for trouble.

10. Notifications

Generated live by `deriveNotifications(data, memberId)` — no stored event log. Every notification has a stable id derived from its source row so the read/dismissed state in `localStorage` persists across re-derives.

Categories (in order they appear in the bell):

Category	Rule	Audience
Overdue tasks	One per overdue task owned by the current member.	Self (urgent prefs).
Tasks due today	One per task owned by current member where <code>dueDate === today</code> .	Self (urgent prefs).
On Fire clients	One per client where <code>status === 'fire'</code> .	All operators (urgent prefs).
Daily digest	One synthesized "X items today" notification per day.	All operators (digest pref).
Time-off pending	One per pending time-off request, capped at 6.	Owner + Admin only.
Time-off decided	Recent decisions (last 14 days), capped at 5.	The requester.

Source: `src/data/deriveNotifications.ts`.

How it should be: Group by client where the underlying signal is identical. "Cobalt Auto · 9 overdue cards" as one notification (clicks to expand the full list) instead of 9 separate notifications. Other categories (digest, time-off, on-fire-client) already aggregate correctly — only overdue + due-today benefit from grouping. The live-derivation model stays; just the surfacing collapses adjacent rows with the same client + same category into one row.

11. External / not-derived-by-us

The Stats tab on the client detail pulls these from third-party integrations, not from our database:

- Ad Spend (Google Ads, Meta Ads, LinkedIn Ads, TikTok Ads...)
- Leads (Hubspot, Salesforce, our own form submissions)
- Blended CPL (computed live: total ad spend / total leads)
- Email Engagement (HubSpot, Mailchimp open rates)
- Channel-level cards (per-integration breakdowns)

The "Live · Synced N min ago" indicator shows the freshness of the latest pull. Our build doesn't store these numbers — every refresh hits the integration's API.

How it should be: Keep the live-fetch model. Don't cache or snapshot — integration data is meant to be fresh, and stale numbers on the Stats tab are worse than no numbers (the AM trusts what's on screen, so a stale number causes worse decisions than a missing one). The "Synced N min ago" indicator already exposes freshness; that's the right contract. Per-integration rate limits should be handled in the integration adapter, not by snapshotting into our database.

How to extend this doc

When a new metric, status, or rollup lands in our build:

- 1 Add it to the relevant section (clients / services / tasks / rollups / capacity).
- 2 Use the same shape: plain-English summary, formal rule, where it surfaces in the UI, source file + function, and a "How it should be" recommendation.
- 3 If it's stored (not derived), note it loudly so the portal knows to add the field.
- 4 If it's derived, paste the rule literally — exact field names, exact comparisons. Ambiguity here causes the portal numbers to drift.

Calculation drift is the most common cause of QA findings between the two builds. Keeping this doc current means we catch divergence in code review rather than four rounds later.